Programming in Python
IDC 212
Lecture - 8

October 6, 2025

Contents

1. Introduction

Introduction

Hierarchy of Python Scientific Libraries

NumPy: Provides core data structures like multi-dimensional arrays and matrices. Essential for numerical computations.

SciPy: Built on NumPy. Adds functionality for optimization, integration, interpolation, and more.

Matplotlib: Plotting and visualization library supporting NumPy/SciPy data.

Pandas: Built on all three (NumPy, SciPy, Matplotlib). Offers powerful data analysis and manipulation tools, especially for tabular data.

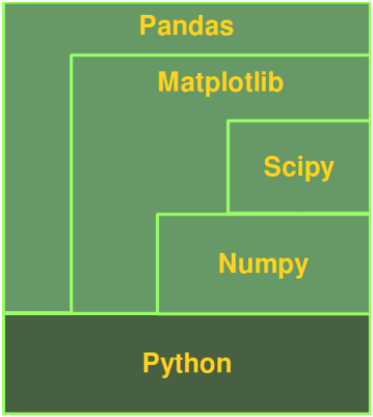


Figure: Relationship among Python Scientific Libraries

Python: A Modern Alternative to MATLAB

- **Python** is increasingly popular among data scientists and engineers.
- It supports powerful libraries like:
  - **NumPy** – for arrays and matrices
  - **SciPy** – for scientific computing
  - **Matplotlib** – for data visualization
  - **Pandas** – for data manipulation
- Python is a general-purpose language with broad applications.

Why Choose Python Over MATLAB or R?

- **Compared to R**:
  - R focuses on statistical computing.
  - Python has a larger and faster-growing community.
- **Compared to MATLAB**:
  - MATLAB is costly.
  - Python is **free and open-source**.
  - Python has similar capabilities through its rich libraries.

Conclusion: Python as a MATLAB Replacement

- Python, in combination with **NumPy, SciPy, Matplotlib, and Pandas**, can serve as a full replacement for MATLAB.
- It is more modern, accessible, and evolving rapidly.
- Supported by a vibrant and large developer community.

Installation and Dependencies

- **NumPy and SciPy are not included in base Python installation.**
- They need to be installed separately using:
  - `pip install numpy`
  - `pip install scipy`
- SciPy **requires** NumPy to be installed first, as it depends on its data structures.

Evolution of NumPy

- NumPy was built by merging two earlier modules:
  - **Numeric**: now obsolete high-performance array module.
  - **Numarray**: complete rewrite of Numeric, also deprecated.
- NumPy combines the strengths of both predecessors.

Importing NumPy

Before using NumPy, we must import it. There are two ways:

```python
import numpy
```

```python
import numpy as np
```

- Second form is most commonly used in practice.
- `np` is a conventional alias for NumPy.

Example: Converting Celsius to Fahrenheit

Step 1: Define the Celsius temperature list

```
cvalues = [20.1, 20.8, 21.9, 22.5, 22.7, 22.3, 21.8, 21.2, 20.9,
    20.1]
```

Step 2: Convert list to NumPy array

```
C = np.array(cvalues)
print(C)
```

Output:

```
[20.1 20.8 21.9 22.5 22.7 22.3 21.8 21.2 20.9 20.1]
```

Fahrenheit Conversion using NumPy

Step 3: Convert Celsius to Fahrenheit using array operations

```
print(C * 9 / 5 + 32)
```

Output:

```
[68.18 69.44 71.42 72.5 72.86 72.14 71.24 70.16 69.62 68.18]
```

Note: Original array `C` remains unchanged.

Comparison: List vs NumPy Array

List comprehension method (less efficient):

```
fvalues = [x * 9/5 + 32 for x in cvalues]
print(fvalues)
```

Output:

```
[68.18, 69.44, 71.42, 72.5, 72.86, 72.14, 71.24000000000001,
    70.16, 69.62, 68.18]
```

- NumPy arrays allow cleaner and faster scalar operations.
- Python lists require explicit loops or comprehensions.

Array Type Check

Check the type of the object `C`:

```
type(C)
```

Output:

```
numpy.ndarray
```

- `ndarray` is the core data structure in NumPy.
- We use the terms "array" and "ndarray" interchangeably.

Creating Arrays with `np.arange()`

`np.arange` is similar to Python's `range`, but it creates a NumPy array instead of a list or iterator.

```
a = np.arange(1, 10)
print(a)
```

Output:

```
[1 2 3 4 5 6 7 8 9]
```

Explanation: This generates numbers starting from 1 up to (but not including) 10 with step size 1. The result is a NumPy array of integers from 1 to 9.

Difference Between `range` and `np.arange`

```
x = range(1, 10)
print(x)
print(list(x))
```

Output:

```
range(1, 10)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Explanation: The built-in `range` returns an iterator object which generates values when needed. To view the contents, you must convert it to a list using `list()`. NumPy's `arange` directly returns an array, which behaves differently and supports vector operations.

Floating Point Steps in `np.arange`

```
x = np.arange(0.5, 10.4, 0.8)
print(x)
```

Output:

```
[ 0.5  1.3  2.1  2.9  3.7  4.5  5.3  6.1  6.9  7.7  8.5  9.3
    10.1]
```

Explanation: This example uses floating point steps of size 0.8. Since floating-point arithmetic is imprecise, the exact values may have small rounding errors. Still, NumPy tries to generate values starting from 0.5 and adds 0.8 repeatedly until it reaches or crosses 10.4.

Floating Point Precision Issues

Be careful with floating point step values:

```
np.arange(12.04, 12.84, 0.08)
```

Output:

```
[12.04 12.12 12.2  12.28 12.36 12.44 12.52 12.6  12.68 12.76
   12.84]
```

Explanation: Floating-point steps can accumulate rounding errors, which may affect the last few values. If you want evenly spaced values and a specific number of points, it is better to use:

```
np.linspace(12.04, 12.84, num=11)
```

`np.linspace()` is more reliable for generating evenly spaced values, especially in plotting.

Force Integer Output from Float Range

You can cast float arange to integers:

```
x = np.arange(0.5, 10.4, 0.8, int)
print(x)
```

Output:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12]
```

Explanation: Here, the float values generated by `arange` are converted to integers using the `dtype=int` argument. This results in all values being truncated (not rounded). The output contains only the integer parts of each value.

Summary

- `np.arange()` is useful for creating numeric sequences.
- Unlike `range()`, it returns a NumPy array with powerful vector operations.
- Float steps in `arange()` may cause precision errors — use `np.linspace()` when exact spacing or number of points is needed.
- You can force integer output by specifying `dtype=int`.

Example 1: Default `linspace`

`np.linspace(1, 10)` generates 50 equally spaced values between 1 and 10 (inclusive).

```python
import numpy as np
print(np.linspace(1, 10))
```

Explanation

- Default `num`=50.
- Step size $= \dfrac{10-1}{50-1} \approx 0.1837$.
- Includes both 1 and 10.

Example 2: Specifying Number of Points

`np.linspace(1, 10, 7)` generates 7 equally spaced values between 1 and 10.

```python
print(np.linspace(1, 10, 7))
# [1.  2.5 4.  5.5 7.  8.5 10. ]
```

Explanation

- 7 points, spacing $= \dfrac{10-1}{7-1} = 1.5$.
- Includes the endpoint 10.

Example 3: Excluding the Endpoint

`np.linspace(1, 10, 7, endpoint=False)` generates 7 values that stop before 10.

```
print(np.linspace(1, 10, 7, endpoint=False))
# [1.      2.2857 3.5714 4.8571 6.1429 7.4286 8.7143]
```

Explanation

- Endpoint excluded.
- Step size $= \dfrac{10-1}{7} \approx 1.2857$.

Example 4: Using `retstep=True`

If `retstep=True`, the function returns both the array and the step size.

```
samples, step = np.linspace(1, 10, retstep=True)
print(step)
# 0.1836734693877551
```

Explanation

- Step size $= \dfrac{10-1}{50-1} \approx 0.1837$.
- Useful when you need the spacing explicitly.

Example 5: `retstep` with Custom `num`

```python
samples, step = np.linspace(1, 10, 20, retstep=True)
print(step)
# 0.47368421052631576
```

Explanation

- 20 points including 10.
- Step size $= \dfrac{10-1}{20-1} \approx 0.4737$.

Example 6: Excluding Endpoint with `retstep`

```python
samples, step = np.linspace(1, 10, 20, endpoint=False, retstep=True)
print(step)
# 0.45
```

Explanation

- 20 points excluding 10.
- Step size $= \dfrac{10-1}{20} = 0.45$.

Zero-Dimensional Array

`np.array(42)` creates a 0D array (scalar).

```
import numpy as np
x = np.array(42)
print("x:", x)
print("Type:", type(x))
print("Dimension:", np.ndim(x))
```

Output: x: 42 Type: <class 'numpy.ndarray'> Dimension: 0
Explanation: - Scalars in NumPy are treated as 0D arrays.

One-Dimensional Arrays

`np.array([...])` creates vectors (1D arrays).

```
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])
V = np.array([3.4, 6.9, 99.8, 12.8])

print("Dimension of F:", np.ndim(F))
print("Dimension of V:", np.ndim(V))
```

Output: F: [1 1 2 3 5 8 13 21] V: [3.4 6.9 99.8 12.8] Dimensions: 1, 1
Explanation: - Both `F` (integers) and `V` (floats) are 1D arrays. - Equivalent to mathematical vectors.

Two-Dimensional Arrays

2D arrays represent matrices.

```
A = np.array([
    [3.4, 8.7, 9.9],
    [1.1, -7.8, -0.7],
    [4.1, 12.3, 4.8]
])

print(A)
print("Dimension:", A.ndim)
```

Output: $\begin{bmatrix} 3.4 & 8.7 & 9.9 \\ 1.1 & -7.8 & -0.7 \\ 4.1 & 12.3 & 4.8 \end{bmatrix}$

Dimension: 2

Explanation: - 2D arrays = matrices. - Shape: rows $\times$ columns.

Three-Dimensional Arrays

3D arrays are like stacks of 2D matrices.

```
B = np.array([
    [[111, 112], [121, 122]],
    [[211, 212], [221, 222]],
    [[311, 312], [321, 322]]
])

print(B)
print("Dimension:", B.ndim)
```

Output (shortened): $\begin{bmatrix} [[111, 112], [121, 122]], \\ [[211, 212], [221, 222]], \\ [[311, 312], [321, 322]] \end{bmatrix}$

Dimension: 3

Explanation: - 3D arrays = tensor (like a cube of numbers). - Shape: (3, 2, 2).

0D array (scalar): shape and ndim

```python
import numpy as np
x = np.array(42)                # scalar -> 0D array
print("x:", x)
print("shape:", x.shape)        # ()
print("ndim:", x.ndim)          # 0
```

Explanation: A scalar is a 0D NumPy array. `shape` is `()` (no axes), `ndim` is 0.

1D array (vector): shape and ndim

```
import numpy as np
F = np.array([1, 2, 3, 4, 5]) # vector -> 1D array
print("F:", F)
print("shape:", F.shape)      # (5,)
print("ndim:", F.ndim)        # 1
```

Explanation: A vector has one axis of length 5. `shape=(5,)` and `ndim=1`.

2D array (matrix): shape and ndim

```
import numpy as np
A = np.array([[1, 2, 3],
              [4, 5, 6]])      # matrix -> 2D array
print("A:\n", A)
print("shape:", A.shape)       # (2, 3)
print("ndim:", A.ndim)         # 2
```

Explanation: A matrix has two axes: rows and columns. `shape=(2,3)`, `ndim=2`.

3D array (tensor): shape and ndim

```
import numpy as np
B = np.array([ [[1,2],[3,4]],
               [[5,6],[7,8]] ])  # tensor -> 3D array
print("B:\n", B)
print("shape:", B.shape)         # (2, 2, 2)
print("ndim:", B.ndim)           # 3
```

Explanation: A 3D tensor is a stack of matrices. `shape=(2,2,2)`, `ndim=3`.

Indexing: 1D arrays (basics)

Goal: Access first and last elements of a vector.

```python
import numpy as np
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])

print(F[0])    # first element
print(F[-1])   # last element
```

Key idea: Zero-based indexing; negative indices count from the end.

Indexing: 2D arrays (two styles)

Goal: Access element in row 1, column 0.

```
A = np.array([[3.4, 8.7,  9.9],
              [1.1, -7.8, -0.7],
              [4.1, 12.3, 4.8]])

print(A[1][0])   # chained indexing (creates a temporary)
print(A[1, 0])   # tuple indexing (preferred, faster)
```

Key idea: Use `A[row, col]`. Chained form works but is less efficient.

Slicing: 1D arrays (start:stop:step)

Goal: Extract subarrays from a 1D array.

```
S = np.array([0,1,2,3,4,5,6,7,8,9])

print(S[2:5])    # indices 2..4
print(S[:4])     # first 4 elements
print(S[6:])     # from index 6 to end
print(S[:])      # all elements (view)
```

Key idea: `start` inclusive, `stop` exclusive; omitted parts default to begin/end.

Slicing: 2D arrays (per-axis ranges)

Goal: Slice by rows and columns independently.

```
A = np.array([[11,12,13,14,15],
              [21,22,23,24,25],
              [31,32,33,34,35],
              [41,42,43,44,45],
              [51,52,53,54,55]])

print(A[:3, 2:])  # rows 0..2, cols 2..end
print(A[3:, :])   # rows 3..end, all cols
print(A[:, 4:])   # all rows, last column
```

Key idea: One slice per axis: `A[row_slice, col_slice]`.

Slicing with steps

Goal: Sample rows/cols at regular intervals.

```
X = np.arange(28).reshape(4, 7)
print(X)
# [[ 0  1  2  3  4  5  6]
#  [ 7  8  9 10 11 12 13]
#  [14 15 16 17 18 19 20]
#  [21 22 23 24 25 26 27]]

print(X[::2, ::3])  # every 2nd row, every 3rd col
print(X[:, ::3])    # all rows, every 3rd col
```

Key idea: `start:stop:step` works independently on each axis.

Omitted axes default to :

Goal: Understand missing slices in higher dims.

```
# For an N-D array, if fewer slices are given, the rest
# are treated as ":" (take all along that axis).

# Example shown for 3D-like selection idea:
# A[1:3, 0:2] == A[1:3, 0:2, :]
```

Key idea: When you provide fewer slices than dimensions, remaining axes are taken entirely.

Views vs. copies: array slicing

Goal: See that slices are *views* (shared memory).

```
A = np.array([0,1,2,3,4,5,6,7,8,9])
S = A[2:6]          # view into A
S[0] = 22
S[1] = 23
print(A)            # [0 1 22 23 4 5 6 7 8 9]
```

Key idea: Modifying a slice modifies the original. Use `A[2:6].copy()` to avoid this.

Lists contrast: slicing makes a copy

Goal: Show Python lists behave differently.

```
lst  = [0,1,2,3,4,5,6,7,8,9]
lst2 = lst[2:6]     # copy
lst2[0] = 22
lst2[1] = 23
print(lst)          # original unchanged
```

Key idea: List slice is a new list; NumPy slice is a view (unless you call `.copy()`).

Checking memory sharing

Goal: Detect whether two arrays may share memory.

```
A = np.arange(12)
B = A.reshape(3, 4)       # just a different view
print(np.may_share_memory(A, B))  # True
B[0,0] = 42
print(A.reshape(3,4))
```

Key idea: `np.may_share_memory` returns True if arrays overlap in memory; reshape usually shares.

Practical tip: make an independent subarray

Goal: Create a slice that won't affect the original.

```
X = np.arange(28).reshape(4, 7)
sub = X[:3, 2:].copy()    # explicit copy
sub[0,0] = -999
print(X[0,2])             # original unchanged
```

Key idea: Use `.copy()` when you need isolation from the source array.

Creating arrays with ones

Goal: Fill arrays with 1s.

```
import numpy as np

E = np.ones((2,3))              # default: float
print(E)

F = np.ones((3,4), dtype=int) # specify int type
print(F)
```

Output: $[[1.1.1.], [1.1.1.]]$ $[[1111], [1111], [1111]]$
Key idea: By default `dtype=float`. Use `dtype=int` for integers.

Creating arrays with zeros

Goal: Fill arrays with 0s.

```
Z = np.zeros((2,4))
print(Z)
```

Output: $[[0.0.0.0.], [0.0.0.0.]]$
Key idea: `np.zeros` works the same way as `np.ones`.

Ones_like and Zeros_like

Goal: Match the shape of an existing array.

```
x = np.array([2,5,18,14,4])

E = np.ones_like(x)    # same shape as x
print(E)

Z = np.zeros_like(x)   # same shape as x
print(Z)
```

Output: [11111] [00000]
Key idea: `*_like` creates arrays with the same shape and type as another.

Empty arrays

Goal: Quickly allocate memory for an array.

```
E = np.empty((2,4))
print(E)
```

Output: $[[0.0.0.0.], [0.0.0.0.]]$ (or arbitrary numbers, depending on memory state)
Key idea: `np.empty` does not initialize values. Entries may look like zeros but can be random garbage in memory.

Views vs. copies: contrast

Goal: Show slice is a view; copy isolates.

```
a = np.arange(10)
v = a[2:6]          # view (shared memory)
c = a[2:6].copy()   # independent

v[0] = 999
print(a)            # affected at a[2]
c[0] = -1
print(a)            # unchanged this time
```

Rule of thumb: When in doubt and you need isolation, call `.copy()`.

Identity array: definition

Identity matrix in linear algebra:

- Square $n \times n$ matrix
- Ones on the main diagonal
- Zeros elsewhere

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In NumPy:

- `np.identity(n)`: always square
- `np.eye(N, M=None, k=0)`: flexible size and diagonal

Using np.identity

```python
import numpy as np

I = np.identity(4)
print(I)

I_int = np.identity(4, dtype=int)
print(I_int)
```

Output:

$$\begin{bmatrix} 1. & 0. & 0. & 0. \\ 0. & 1. & 0. & 0. \\ 0. & 0. & 1. & 0. \\ 0. & 0. & 0. & 1. \end{bmatrix}$$

Key idea: `np.identity(n)` creates an $n \times n$ identity matrix.

Using np.eye

```
E = np.eye(5, 8, k=1, dtype=int)
print(E)
```

Output:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Key idea:

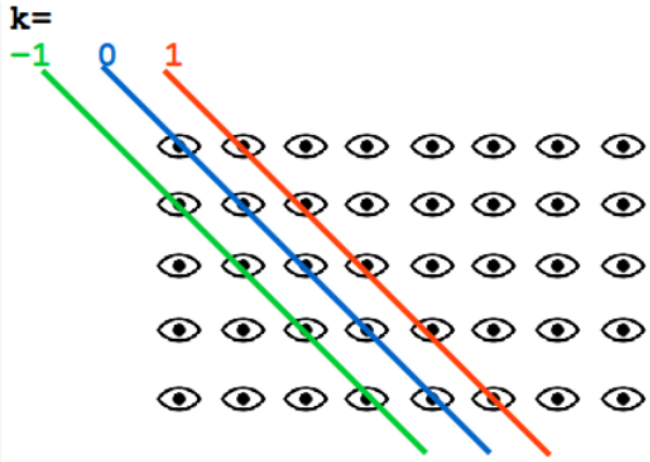
- `N=5`, `M=8` $\Rightarrow$ 5 rows, 8 columns
- `k=1` $\Rightarrow$ ones on diagonal above main

Eye function: shifting diagonals

```
np.eye(4, 4, k=0)   # main diagonal
np.eye(4, 4, k=1)   # upper diagonal
np.eye(4, 4, k=-1)  # lower diagonal
```

Diagram:
- $k = 0 \rightarrow$ main diagonal (blue) - $k = 1 \rightarrow$ upper diagonal (red) - $k = -1 \rightarrow$ lower diagonal (green)

Eye function: shifting diagonals



Creating a simple dtype and using it

```python
import numpy as np

i16 = np.dtype(np.int16)
print(i16)

lst = [[3.4, 8.7, 9.9],
       [1.1, -7.8, -0.7],
       [4.1, 12.3, 4.8]]

A = np.array(lst, dtype=i16)
print(A)
```

Key idea: Casting floats to `int16` truncates values.

Structured dtype with one field

```python
import numpy as np

dt = np.dtype([('density', np.int32)])
x = np.array([(393,), (337,), (256,)], dtype=dt)

print(x)
print(repr(x))            # internal representation

print(x['density'])       # access field
```

Key idea: Structured dtypes allow column access by name.

Structured dtype with multiple fields

```
dt = np.dtype([('country', 'S20'),
               ('density', 'i4'),
               ('area', 'i4'),
               ('population', 'i4')])

population_table = np.array([
  ('Netherlands', 393, 41526, 16928800),
  ('Belgium',     337, 30510, 11007020),
  ('United Kingdom', 256, 243610, 62262000),
], dtype=dt)

print(population_table[:2])
print(population_table['density'])
print(population_table['country'])
```

Unicode strings in structured arrays

```
dt = np.dtype([('country', np.unicode_, 20),
               ('density', 'i4'),
               ('area', 'i4'),
               ('population', 'i4')])

population_table = np.array([
  ('Netherlands', 393, 41526, 16928800),
  ('Belgium',     337, 30510, 11007020),
  ('United Kingdom', 256, 243610, 62262000),
], dtype=dt)

print(population_table[:2])
```

Key idea: `np.unicode_` stores strings as Unicode (no `b"` prefix).

Input and Output of structured arrays

```
np.savetxt("population_table.csv",
           population_table,
           fmt="%s;%d;%d;%d",
           delimiter=";")

dt = np.dtype([('country', np.unicode_, 20),
               ('density', 'i4'),
               ('area', 'i4'),
               ('population', 'i4')])

x = np.genfromtxt("population_table.csv",
                  dtype=dt,
                  delimiter=";")

print(x)
```

Key idea: Save structured arrays with `savetxt`, reload with `genfromtxt`.

Problem: arange and linspace

Q:

1. Create an array of numbers from 5 to 50 with step size 5 using `np.arange`.
2. Create 20 equally spaced points between 0 and 2π using `np.linspace`. Then compute the sine of these values using `np.sin`.

Solution (a): Using np.arange

```python
import numpy as np

a = np.arange(5, 55, 5)
print(a)
```

Output:

```
[ 5 10 15 20 25 30 35 40 45 50]
```

Explanation: `np.arange(start, stop, step)` Generates numbers from 5 up to (but not including) 55, step 5.

Solution (b): Using np.linspace and np.sin

```python
import numpy as np

b = np.linspace(0, 2*np.pi, 20)
s = np.sin(b)

print(b)
print(s)
```

Output (rounded):

```
[0.      0.3307 0.6614 ... 5.9525 6.2832]
[0.      0.3256 0.6131 ... -0.3256 -0.0000]
```

Explanation: - `np.linspace(0, 2*np.pi, 20)` $\rightarrow$ 20 points from 0 to 2π. - `np.sin(b)` $\rightarrow$ sine of each value.

Problem: Ones, Zeros, and Empty

Q:

1. Create a 3×4 array of ones (int type).
2. Create a 2×5 array of zeros (float type).
3. Use `np.ones_like` and `np.zeros_like` on an existing array `x = np.array([5,10,15])`.
4. Use `np.empty((3,3))`. What do you observe?

Solution (a): 3×4 Ones

```python
import numpy as np

a = np.ones((3,4), dtype=int)
print(a)
```

Output:

```
[[1 1 1 1]
 [1 1 1 1]
 [1 1 1 1]]
```

Solution (b): 2×5 Zeros

```python
b = np.zeros((2,5), dtype=float)
print(b)
```

Output:

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

Solution (c): ones_like and zeros_like

```python
x = np.array([5, 10, 15])

c1 = np.ones_like(x)
c2 = np.zeros_like(x)

print(c1)
print(c2)
```

Output:

```
[1 1 1]
[0 0 0]
```

Solution (d): Empty array

```
d = np.empty((3,3))
print(d)
```

Output (may vary):

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Explanation: `np.empty` allocates memory but does not initialize values.

Problem: Structured Arrays – Population Table

Q:

- Create a structured array with fields:
  - country (str, length 20)
  - density (int)
  - area (int)
  - population (int)
- Insert sample data for 3 countries.
- (a) Print the first two rows.
- (b) Print all country names.
- (c) Print area for the last country only.

Solution: Structured Arrays

```python
import numpy as np

dt = np.dtype([
    ('country', 'U20'),
    ('density', 'i4'),
    ('area', 'i4'),
    ('population', 'i8')
])

population_table = np.array([
  ('Netherlands', 393, 41526, 16928800),
  ('Belgium',     337, 30510, 11007020),
  ('United Kingdom', 256, 243610, 62262000)
], dtype=dt)
```

Solution: Queries

```python
# a) First two rows
print(population_table[:2])

# b) All country names
print(population_table['country'])

# c) Area of last country
print(population_table[-1]['area'])
```

Output:

```
[('Netherlands', 393,  41526, 16928800)
 ('Belgium',     337,  30510, 11007020)]
['Netherlands' 'Belgium' 'United Kingdom']
243610
```

Problem: Indexing and Slicing

Q: Given the array A, perform the following operations.

```
A = np.array([
 [11, 12, 13, 14, 15],
 [21, 22, 23, 24, 25],
 [31, 32, 33, 34, 35],
 [41, 42, 43, 44, 45],
 [51, 52, 53, 54, 55]
])
```

1. Extract the subarray of the first 3 rows and last 3 columns.
2. Extract the last two rows.
3. Extract the first column only.
4. Slice every 2nd row and 2nd column.

Solution (a): First 3 rows, last 3 columns

```python
print(A[:3, 2:])
```

Output:

```
[[13 14 15]
 [23 24 25]
 [33 34 35]]
```

Solution (b): Last two rows

```python
print(A[-2:, :])
```

Output:

```
[[41 42 43 44 45]
 [51 52 53 54 55]]
```

Solution (c): First column only

```python
print(A[:, 0])
```

Output:

```
[11 21 31 41 51]
```

Solution (d): Every 2nd row and 2nd column

```python
print(A[::2, ::2])
```

Output:

```
[[11 13 15]
 [31 33 35]
 [51 53 55]]
```

Thank You